

Integración SDK Nexgo con Flutter y Código Nativo Java – Documentación Técnica

1. Estructura del Proyecto y Configuración Inicial

Esta sección documenta la estructura inicial requerida para integrar una aplicación Flutter con un SDK nativo en Java utilizando archivos AAR y comunicación mediante Platform Channels.

1.1 Ubicación de los archivos del SDK nativo

El SDK de Nexgo está compuesto por dos archivos. aar que deben ubicarse dentro del directorio del proyecto Flutter en la siguiente ruta:

android/app/libs/

- nexgo-smartpos-sdk-v3.08.002_20240410.aar
- smartconnect_new_version.aar

1.2 Configuración del build.gradle.kts

Luego de agregar los archivos AAR en el directorio correspondiente, deben ser incluidos en el archivo android/app/build.gradle.kts para que el proyecto pueda compilarlos adecuadamente.

Configuración utilizada:

- Inclusión del plugin de Flutter, Kotlin y el plugin de Android.
- Declaración del namespace del proyecto.
- Activación de Java 11 como versión de compilación.
- Inclusión de las dependencias para los archivos AAR mediante fileTree y files().

1.3 Dependencias del SDK

El proyecto necesita incluir algunos componentes adicionales como AndroidX, Gson y Multidex. Las dependencias del SDK son las siguientes:

- Kotlin Standard Library
- Multidex
- AndroidX AppCompat
- ConstraintLayout
- Gson
- Annotation

1.4 Estructura Java y archivos clave

Dentro del directorio main/java se debe estructurar el código nativo siguiendo el espacio de nombres requerido por el SDK. En este caso se crea el paquete:

- main/java/cn/nexgo/smartconnect/model/data/

Dentro de este paquete se coloca el archivo Constant.java que contiene las constantes necesarias para las transacciones del dispositivo.

Además, en el paquete principal de la app Flutter main/java/com/example/disglobal_sdk_demo_for_flutter/ se encuentra el archivo MainActivity.java encargado de:

- Registrar el canal de comunicación con Flutter.
- Implementar la función bindService necesaria para el SDK.
- Implementar las funciones doTransaction, printReceipt y printerTest.

2. Documento Técnico

Comunicación Flutter ↔ Java y Funciones Principales

2.1 Comunicación entre Flutter y Código Nativo (Platform Channels)

Esta sección detalla la implementación del canal nativo utilizado para comunicar la aplicación Flutter con el SDK Java de Nexgo. La integración se basa en un MethodChannel llamado:

```
static const platform = MethodChannel('nexgo_service');
```

Este canal permite enviar solicitudes desde Flutter hacia Java y recibir las respuestas correspondientes.

2.1 Archivo Constant.java – Constantes del SDK

Este archivo contiene los códigos numéricos utilizados por el SDK Nexgo para definir el tipo de transacciones.

Constante	Valor	Descripción
TRANS_TYPE_SALE	1	Compra normal
TRANS_TYPE_VOID	2	Anulación de una transacción
TRANS_TYPE_EXTRA	3	Transacciones especiales
TRANS_TYPE_SETTLEMENT	4	Proceso de cierre (batch)
TRANS_TYPE_REPROT	5	Reporte de transacciones
TRANS_TYPE_TIP	6	Propinas
TRANS_TYPE_TEST	7	Transacción de prueba
MODE_SETUP_PAGE	10	Modo de configuración

Estas constantes son esenciales para enviar correctamente el tipo de operación al dispositivo.

2.2 MainActivity.java – Puente principal Flutter ↔ SDK Nexgo

El archivo MainActivity.java es responsable de:

1. Registrar el canal de comunicación.
2. Recibir llamadas desde Flutter.
3. Ejecutar acciones nativas sobre el SDK Nexgo.
4. Retornar resultados en formato JSON hacia Flutter.

2.2.1 MethodChannel y Registro

Dentro de configureFlutterEngine, se crea el MethodChannel nexgo_service y se asocian los métodos disponibles:

Método	Descripción
bindService	Conecta el terminal con el servicio SmartConnect del SDK. Debe ejecutarse antes de cualquier transacción.
doTransaction	Ejecuta una transacción de compra, cierre o prueba según el código enviado.
printReceipt	Imprime un comprobante real, con todos los datos del pago.
printerTest	Imprime un ticket de prueba (logo, items, fecha, monto).

2.2.2 bindService() – Conexión con SmartConnect

Este método realiza el bind del servicio del SDK Nexgo, requisito para ejecutar transacciones.

✓ Sin este bind, ninguna transacción funcionará.

✓ Retorna true si se conectó correctamente.

Se conecta al servicio:

cn.nexgo.inbas.smartconnect.SmartConnectService

Una vez conectado, se obtiene la instancia de:

ISmartconnectService smartService

que permite ejecutar cualquier transacción con el pinpad.

2.2.3 doTransaction() – Ejecutar Transacciones

Este método recibe parámetros desde Flutter:

- amount → Monto en formato string
- cardholderId → Identificación del titular
- waiterNum → Número de mesonero (si aplica)
- referenceNo → Número único asignado a la transacción
- transType → Tipo de transacción (basado en Constant.java)

Internamente:

1. Construye el objeto TransactionRequestEntity.
2. Envía la solicitud a smartService.transactionRequest.
3. Recibe la respuesta mediante el listener ITransactionResultListener.
4. Serializa la respuesta a JSON usando Gson.
5. Envía el JSON completo de vuelta a Flutter.

Comportamientos:

- Si `trx.getResult() == 0`, se considera transacción **aprobada** → `result.success(json)`
- Si no, se considera **rechazada** → `result.error(json, json, null)`

2.2.4 printReceipt() – Impresión de comprobante real

Este método se encarga de imprimir un recibo real para el cliente. Recibe desde Flutter todos los datos necesarios:

- Nombre del cliente
- Cédula
- Monto
- Contrato
- Lote
- Terminal
- Serial
- Trace
- Fecha/Hora

El flujo del método es:

1. Inicializa el printer con **initPrinter()**.
2. Imprime textos alineados usando `appendPrnStr`.
3. Ejecuta `startPrint` para enviar el trabajo a la impresora.
4. Retorna `printed` si la impresión fue exitosa.

2.2.5 printerTest() – Ticket de prueba

Este método imprime un ticket con:

- ✓ Logo corporativo (cargado desde Flutter assets)
- ✓ Fecha y hora
- ✓ Algunos items de ejemplo
- ✓ Total
- ✓ Mensaje final

Es ideal para probar la impresora sin procesar una transacción real.

2.3 nexgo_functions.dart – Comunicación desde Flutter

Este archivo contiene las clases que envían solicitudes a Java mediante MethodChannel.

2.3.1 DoTransaction – Procesar compras y cierres

Funciones principales:

- **bindService()** Solicita a Java conectar el servicio SmartConnect. Debe ejecutarse una vez al iniciar la app o antes de cualquier transacción.
- **doTransaction()** Ejecuta una transacción normal. Con los parámetros:
 - amount
 - cardholderId
 - waiterNum
 - referenceNo
 - transType

Recibe JSON → lo convierte a **RecordResponse**.

Maneja:

- success en result.success
 - error en result.error (el JSON viene dentro del error.message)
- **doTransactionSettlement()** Versión simplificada usada para realizar un cierre de lote. Por defecto envía:
 - amount = ''
 - cardholderId = ''
 - transType = 4

Convierte la respuesta en un modelo `SettlementResponse`.

2.3.2 PrinterPos – Impresión desde Flutter

Funciones Principales:

- **imprimirPos()** Llama a printReceipt en Java para imprimir un comprobante real.

Envía:

- fullName
- ciClient
- amount
- contrato
- referencia
- fecha/hora
- lote

- afiliado
 - terminal
 - serial
 - trace
- **imprimirTestPos()** Llama al método Java `printerTest` para imprimir un ticket de prueba.
Envía:
 - fecha/hora

2.4 Ejecución de Funciones del SDK Desde los Botones (Flutter UI)

En esta sección se detalla cómo llamar las funciones principales del SDK (transacción, impresión y cierre) desde los botones en las pantallas de ejemplo de Flutter.

Estas implementaciones corresponden a las pantallas:

- **SaleScreen** → Procesar una compra
- **PrinterScreen** → Imprimir comprobante de prueba
- **SettlementScreen** → Ejecutar un cierre o “settlement”

2.4.1 Ejecutar una Compra (SaleScreen)

En esta pantalla se instancia la clase:

final transaccion = DoTransaction();

La lógica del botón **PAGAR** sigue tres pasos:

- 1- Inicializar el servicio Nexgo (`bindService`)
await transaccion.bindService();
await Future.delayed(const Duration(milliseconds: 500));
 Este delay de 500 ms asegura que el servicio esté completamente listo antes de continuar.
- 2- Ejecutar la transacción (`doTransaction`) Se envía el monto, la cédula del titular y el tipo de transacción:

```
final value = await transaccion.doTransaction(
  transformarAmountaEntero(amount.text), // monto convertido
  numberDocument.text,                  // cédula
  '',
  '',
  1,                                     // 1 = compra
);
```

- 3- Manejar la respuesta (aprobado o fallido)
 El valor retornado puede venir como:

- RecordResponse
- String con JSON
- Map con JSON

Por eso se parsea de manera segura:

```
late RecordResponse result;

if (value is RecordResponse) {
  result = value;
} else if (value is String) {
  result = RecordResponse.fromJson(jsonDecode(value));
} else if (value is Map) {
  result = RecordResponse.fromJson(Map<String, dynamic>.from(value));
}
```

Luego se valida el resultado:

```
if (result.result == 0) {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => TransactionApprovedScreen(
        recordResponse: result,
        document: numberDocument.text,
        amountReintentar: transformarAmountaEntero(amount.text),
      ),
    ),
  );
} else {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => TransactionFailedScreen(
        recordResponse: result,
        amountReintentar: transformarAmountaEntero(amount.text),
        document: numberDocument.text,
      ),
    ),
  );
}
```


2.4.2 Imprimir Ticket de Prueba (PrinterScreen)

Para imprimir un comprobante de prueba se usa la clase:

```
final printer = PrinterPos();
```

Y en el botón IMPRIMIR:

```
await printer.imprimirTestPos(  
    DateFormat('dd/MM/yyyy').format(DateTime.now()),  
    DateFormat().add_Hms().format(DateTime.now()),  
);
```

Este método llama internamente a Java **printerTest()**.

2.4.3 Hacer un Cierre (SettlementScreen)

Para realizar un cierre (settlement), se utiliza:

```
final transaccion = DoTransaction();
```

1- Inicializamos el servicio:

```
await transaccion.bindService();  
await Future.delayed(const Duration(milliseconds: 500));
```

2- Ejecutamos la operación de cierre:

```
final value = await transaccion.doTransactionSettlement();
```

Este método ya configura:

- amount = "0"
- cardholderId = "0"
- transType = 4 (cierre)

3- Parseo seguro:

```
late SettlementResponse result;  
  
if (value is SettlementResponse) {  
    result = value;  
} else if (value is String) {  
    result = SettlementResponse.fromJson(jsonDecode(value));  
}
```

```

} else if (value is Map) {
  result = SettlementResponse.fromJson(Map<String, dynamic>.from(value));
}

```

4- Logica final de cierre:

```

if (result.result == 0) {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => SettlementApproved(settlementResponse: result),
    ),
  );
} else {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => SettlementFailed(settlementResponse: result),
    ),
  );
}

```

6. Modelos de Respuesta JSON

A continuación, se agregan los modelos completos solicitados por el usuario.

6.1 Modelo RecordResponse

```

import 'dart:convert';

RecordResponse recordResponseFromJson(String str) =>
RecordResponse.fromJson(json.decode(str));

String recordResponseToJson(RecordResponse data) =>
json.encode(data.toJson());

class RecordResponse {
  String? rrn;
  int accountType;
  String amount;
  String batchNum;
  String date;
  String deviceSerial;
  int errorCode;
  String merchantId;
}

```

```

String referenceNumber;
String responseCode;
String responseMessage;
int result;
String terminalId;
String time;
String tipAmount;
String traceNumber;
int transType;

RecordResponse({
    this.rrn = '0',
    required this.accountType,
    required this.amount,
    required this.batchNum,
    required this.date,
    required this.deviceSerial,
    required this.errorCode,
    required this.merchantId,
    required this.referenceNumber,
    required this.responseCode,
    required this.responseMessage,
    required this.result,
    required this.terminalId,
    required this.time,
    required this.tipAmount,
    required this.traceNumber,
    required this.transType,
});

factory RecordResponse.fromJson(Map<String, dynamic> json) =>
RecordResponse(
    rrn: json["RRN"],
    accountType: json["accountType"],
    amount: json["amount"],
    batchNum: json["batchNum"],
    date: json["date"],
    deviceSerial: json["deviceSerial"],
    errorCode: json["errorCode"],
    merchantId: json["merchantID"],
    referenceNumber: json["referenceNumber"],
    responseCode: json["responseCode"],
    responseMessage: json["responseMessage"],
    result: json["result"],
    terminalId: json["terminalID"],

```

```

        time: json["time"],
        tipAmount: json["tipAmount"],
        traceNumber: json["traceNumber"],
        transType: json["transType"],
    );

    Map<String, dynamic> toJson() => {
        "RRN": rrn,
        "accountType": accountType,
        "amount": amount,
        "batchNum": batchNum,
        "date": date,
        "deviceSerial": deviceSerial,
        "errorCode": errorCode,
        "merchantID": merchantId,
        "referenceNumber": referenceNumber,
        "responseCode": responseCode,
        "responseMessage": responseMessage,
        "result": result,
        "terminalID": terminalId,
        "time": time,
        "tipAmount": tipAmount,
        "traceNumber": traceNumber,
        "transType": transType,
    };
}

```

6.2 Modelo SettlementResponse

```

import 'dart:convert';

SettlementResponse settlementResponseFromJson(String str) =>
SettlementResponse.fromJson(json.decode(str));

String settlementResponseToJson(SettlementResponse data) =>
json.encode(data.toJson());

class SettlementResponse {
    String? creditBatchNo;
    String? debitBatchNo;
    String? extraBatchNo;
    String? rrn;
    int? accountType;
    String? date;
    String? deviceSerial;
    int? errorCode;
}

```

```
String? merchantId;
String? referenceNumber;
String? responseCode;
String? responseMessage;
int? result;
String? terminalId;
String? time;
String? tipAmount;
String? totalCreditCardRefund;
String? totalCreditCardSale;
String? totalDebitCardRefund;
String? totalDebitCardSale;
String? totalExtraRefund;
String? totalExtraSale;
String? traceNumber;
int? transType;

SettlementResponse({
  this.creditBatchNo,
  this.debitBatchNo,
  this.extraBatchNo,
  this.rrn,
  this.accountType,
  this.date,
  this.deviceSerial,
  this.errorCode,
  this.merchantId,
  this.referenceNumber,
  this.responseCode,
  this.responseMessage,
  this.result,
  this.terminalId,
  this.time,
  this.tipAmount,
  this.totalCreditCardRefund,
  this.totalCreditCardSale,
  this.totalDebitCardRefund,
  this.totalDebitCardSale,
  this.totalExtraRefund,
  this.totalExtraSale,
  this.traceNumber,
  this.transType,
});
```

```

    factory SettlementResponse.fromJson(Map<String, dynamic> json) =>
SettlementResponse(
    creditBatchNo: json["CreditBatchNo"],
    debitBatchNo: json["DebitBatchNo"],
    extraBatchNo: json["ExtraBatchNo"],
    rrn: json["RRN"],
    accountType: json["accountType"],
    date: json["date"],
    deviceSerial: json["deviceSerial"],
    errorCode: json["errorCode"],
    merchantId: json["merchantID"],
    referenceNumber: json["referenceNumber"],
    responseCode: json["responseCode"],
    responseMessage: json["responseMessage"],
    result: json["result"],
    terminalId: json["terminalID"],
    time: json["time"],
    tipAmount: json["tipAmount"],
    totalCreditCardRefund: json["totalCreditCardRefund"],
    totalCreditCardSale: json["totalCreditCardSale"],
    totalDebitCardRefund: json["totalDebitCardRefund"],
    totalDebitCardSale: json["totalDebitCardSale"],
    totalExtraRefund: json["totalExtraRefund"],
    totalExtraSale: json["totalExtraSale"],
    traceNumber: json["traceNumber"],
    transType: json["transType"],
);

```

```

Map<String, dynamic> toJson() => {
    "CreditBatchNo": creditBatchNo,
    "DebitBatchNo": debitBatchNo,
    "ExtraBatchNo": extraBatchNo,
    "RRN": rrn,
    "accountType": accountType,
    "date": date,
    "deviceSerial": deviceSerial,
    "errorCode": errorCode,
    "merchantID": merchantId,
    "referenceNumber": referenceNumber,
    "responseCode": responseCode,
    "responseMessage": responseMessage,
    "result": result,
    "terminalID": terminalId,
    "time": time,
    "tipAmount": tipAmount,

```

```
    "totalCreditCardRefund": totalCreditCardRefund,  
    "totalCreditCardSale": totalCreditCardSale,  
    "totalDebitCardRefund": totalDebitCardRefund,  
    "totalDebitCardSale": totalDebitCardSale,  
    "totalExtraRefund": totalExtraRefund,  
    "totalExtraSale": totalExtraSale,  
    "traceNumber": traceNumber,  
    "transType": transType,  
  };  
}
```

7. Nota Adicional – Recomendación Importante

Para mejorar la mantenibilidad del proyecto, se recomienda:

→ **Crear una carpeta exclusiva de modelos en:**

lib/models/

→ **Separar cada modelo en su propio archivo:**

- record_response.dart

- settlement_response.dart

→ **Agregar un archivo indexador opcional:**

lib/models/models.dart

Que exporte todos los modelos para importarlos de forma más limpia, esta organización hará la integración más escalable y profesional.